

Software Test Plan (STP)

Hostel Management System

Project: Hostel Management System

Team: Team 5

Version: 1.0

Date: 16-09-2026

Status: Draft for review

Team Members

Name	SRN
Arjav Chandranath	PES2UG24CS080
Arjun Mohan Gowda	PES2UG24CS082
Armaan Ali	PES2UG24CS083
Arnav Sharath Hanchanur	PES2UG24CS084

1. Introduction

1.1 Purpose

This document is the Software Test Plan (STP) for the Hostel Management System (HMS) version 1.0. It states what will be tested, what will not, how testing will be carried out, who is responsible for each part, what environment and tools are needed, and the criteria that decide whether testing may start, must stop, or is finished. Every planned test traces back to a requirement in the HMS SRS v1.0, so that requirement coverage can be shown rather than assumed.

The plan follows the IEEE 829 test plan structure used in the course template. It is written for the development team (who are also the testers), the Course Coordinator and evaluators, and any future student team that maintains the system.

1.2 Scope

Testing covers the six feature groups specified in SRS Section 4 and the non-functional and security requirements in Sections 5 and 5.1:

- Authentication, profile management and role-based access control (HMS-F-001 to HMS-F-005)
- Room allocation, reallocation and hostel inventory management (HMS-F-010 to HMS-F-015)
- Hostel fee configuration, online payment, receipts and reconciliation (HMS-F-020 to HMS-F-024)
- Maintenance and complaint tracking (HMS-F-030, HMS-F-031)
- Administrative reporting and export (HMS-F-040, HMS-F-041)
- Non-functional requirements HMS-NF-001 to HMS-NF-006 and security requirements HMS-SR-001 to HMS-SR-005

Testing is carried out at four levels - unit, integration, system and acceptance - on the team's own development and staging deployments. Items excluded from testing, and the reasons for excluding them, are listed in Section 4.

1.3 References

Ref	Document	Version / Date
R1	Hostel Management System - Software Requirements Specification (baseline for all traceability)	v1.0, 04-09-2026
R2	Jackfruit - Software Engineering Project Guidelines, UE24CS341A	2026
R3	IEEE 829 - Standard for Software Test Documentation (structure of this document)	2008

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
HMS	Hostel Management System, the software under test
STP / RTM	Software Test Plan / Requirements Traceability Matrix
FR / NFR / SR	Functional requirement / non-functional requirement / security requirement (HMS-F-###, HMS-NF-###, HMS-SR-###)
UT / IT / ST / SEC / PERF	Test case prefixes for unit, integration, system, security and performance test cases
RBAC	Role-Based Access Control (Student, Warden/Admin, Technician sub-role)
BVA / EP	Boundary Value Analysis / Equivalence Partitioning, the main black-box design techniques used here
IDOR	Insecure Direct Object Reference - reaching another user's record by changing an ID in the URL
p95	The 95th percentile of a measurement; 95% of observations are at or below it
SUT	System Under Test
UAT	User Acceptance Testing
Severity 1-4	Defect impact bands defined in Section 14.3

2. Test Items

The following items are delivered for testing. Each is a Django application or an external interface identified in SRS Sections 2.1 and 3.3.

Test item	Description	Source of truth
AuthModule	Registration, login, session handling, OTP password reset, profile update, RBAC enforcement	SRS 4.1
AllocationModule	Hostel block/floor/room inventory, capacity rules, allocation, reallocation, room-change workflow	SRS 4.2
FeeModule	Fee structure configuration, fee statement, payment initiation, webhook handling, receipts, reconciliation	SRS 4.3
ComplaintModule	Complaint creation with attachment, ticket IDs, status workflow, notifications	SRS 4.4
ReportingModule	Occupancy, vacancy, fee-collection and defaulter reports, CSV/PDF export	SRS 4.5
Audit & logging component	Immutable audit trail for administrative actions and fee transactions	SRS 5.1.2 (HMS-SR-004)
Web UI (Django templates + Bootstrap)	Student dashboard and Admin/Warden dashboard, responsive layouts	SRS 3.1
Optional REST API layer (DRF)	Allocation, fee and complaint endpoints - tested only if implemented in Sprint 3	SRS 3.3
Database schema (PostgreSQL)	Constraints, referential integrity, transaction behaviour for allocation and payments	SRS 2.5, HMS-F-015

Test item	Description	Source of truth
External interface: payment gateway (sandbox)	HMS side of the integration only - order creation, redirect, webhook verification	SRS 3.3
External interface: email/SMS gateway	HMS side of the integration only - OTP and notification send calls, mocked in test	SRS 3.3
Deployment configuration	Gunicorn/Nginx TLS settings, Django security settings, Docker compose definition	SRS 3.4, HMS-SR-001

3. Features to be Tested

All 20 functional requirements, 6 non-functional requirements and 5 security requirements of SRS v1.0 are in scope. The "anchor test case" column carries the test case identifiers already named in the SRS acceptance criteria; each anchor is expanded into 8-10 detailed unit, integration and system test cases in the separate Test Case Specification, using the numbering convention given in Section 13.1.

3.1 Functional Requirements

Req ID	Requirement (short)	Levels	Pri.	Anchor test case(s)
HMS-F-001	Register and log in with institute email/roll number; role assigned as Student, Warden or Admin	UT, IT, ST	High	TC-Auth-01
HMS-F-002	Student updates own profile (contact, address, emergency contact)	UT, ST	Med	TC-Auth-02
HMS-F-003	Password reset via OTP to registered email, 10-minute expiry	IT, ST	Med	TC-Auth-03
HMS-F-004	Inactive session terminated after 20 minutes; re-authentication required	ST	High	TC-Auth-04
HMS-F-005	RBAC: Students cannot reach Admin/Warden views and vice versa; 403 returned and logged	UT, IT, SEC	High	TC-Auth-05, TC-Sec-04
HMS-F-010	Student views own room allocation and roommate details	UT, ST	High	TC-Alloc-01
HMS-F-011	Admin/Warden creates, views, updates and deactivates student records	UT, IT	High	TC-Alloc-02
HMS-F-012	Allocate or reallocate a room by availability, gender policy and stated preference; occupancy updates atomically	UT, IT, ST	High	TC-Alloc-03
HMS-F-013	Student raises room-change request; Admin approves or rejects, transitions logged with timestamp	IT, ST	Med	TC-Alloc-04
HMS-F-014	Admin manages blocks, floors, rooms and capacity/type (single/shared)	UT, IT	High	TC-Alloc-05
HMS-F-015	Allocation beyond configured room capacity is rejected with a clear error and logged	UT, IT (DB constraint)	High	TC-Alloc-06

Req ID	Requirement (short)	Levels	Pri.	Anchor test case(s)
HMS-F-020	Student views itemised fee statement: dues, paid amount, due date	UT, ST	High	TC-Fee-01
HMS-F-021	Student pays hostel fees online through the sandbox payment gateway	IT, ST	High	TC-Fee-02
HMS-F-022	Admin configures fee structure per academic term and room type without altering paid records	UT, IT	Med	TC-Fee-03
HMS-F-023	PDF payment receipt generated and downloadable after each successful transaction	IT, ST	Med	TC-Fee-04
HMS-F-024	Admin reconciles payments against the gateway ledger and flags defaulters	IT, ST	High	TC-Fee-05
HMS-F-030	Student raises a maintenance complaint with category, description and optional photo; unique ticket ID, status Open	UT, ST	Med	TC-Maint-01
HMS-F-031	Admin/technician moves complaint Open to In Progress to Resolved; student notified, changes timestamped	IT, ST	Med	TC-Maint-02
HMS-F-040	Admin generates occupancy, vacancy and fee-collection reports for a selected date range	IT, ST	Med	TC-Rep-01
HMS-F-041	Reports exported in CSV and PDF, matching on-screen data	ST	Low	TC-Rep-02

3.2 Non-Functional Requirements

Req ID	Requirement and measurement	Method	Pri.	Anchor test case(s)
HMS-NF-001	95% of page loads and form submissions complete within 2 s at up to 200 concurrent users (p95 <= 2 s)	Load test (Locust)	High	TC-Perf-01
HMS-NF-002	99.5% availability per semester, excluding planned maintenance	Partial: 4-hour soak test plus review of uptime monitor and restart policy - see Section 4	High	TC-Ops-02
HMS-NF-003	Student personal and payment data protected; no PAN/card data stored on HMS servers	Schema and code review, log inspection	High	TC-Sec-01
HMS-NF-004	Fee-transaction and allocation audit logs retained at least 3 years	Partial: inspect retention configuration and demonstrate archive/restore on seeded data - see Section 4	High	TC-Ops-01
HMS-NF-005	Responsive from 360 px width; WCAG 2.1 AA colour contrast and keyboard navigation where practical	Manual audit at 360/768/1366 px, axe DevTools, keyboard-only pass	Med	TC-UX-01
HMS-NF-006	Supports 5,000 student records and 2,000 rooms without redesign; modular Django apps per feature	Scale test on seeded data plus module-structure review	Med	TC-Scale-01

3.3 Security Requirements

These are planned in outline here and specified in detail in Section 5.1.

Req ID	Requirement (short)	Pri.	Anchor test case(s)
HMS-SR-001	TLS 1.2+ mandatory for all client-server traffic; HTTP redirected to HTTPS	High	TC-Sec-02
HMS-SR-002	Passwords stored with a salted industry-standard hash (PBKDF2/Argon2), never plaintext	High	TC-Sec-03
HMS-SR-003	RBAC enforced at both view/template and API layer, not the UI alone	High	TC-Sec-04
HMS-SR-004	All administrative actions recorded in an immutable audit log with user, timestamp and action	High	TC-Sec-05
HMS-SR-005	Payments processed exclusively through a PCI-DSS compliant gateway; no raw card/CVV data in HMS	High	TC-Sec-06

4. Features Not to be Tested

Each exclusion below is recorded so that the gap is deliberate and visible to evaluators, rather than an omission. Excluded requirements appear in the RTM (Section 13) with status N and a justification.

Excluded item	Reason for exclusion
Django framework, Django ORM and PostgreSQL engine internals	Upstream components with their own test suites; HMS tests only its own models, queries and constraints
Payment gateway card capture, 3-D Secure and settlement processing	Runs entirely on gateway infrastructure. Only the HMS side of the interface - order creation, redirect, webhook verification, ledger reconciliation - is tested, in sandbox mode
Email/SMS delivery infrastructure	Third-party service. HMS is tested up to the send call; delivery is simulated with a local SMTP catcher (MailHog)
Biometric access control, CCTV, mess/canteen management, full ERP integration	Explicitly out of scope in SRS Section 1.2
HMS-NF-002 (99.5% availability measured over a full semester)	The measurement window is longer than the project itself. Replaced by a 4-hour soak test, review of the Gunicorn/Nginx restart policy and inspection of the uptime monitor configuration (TC-Ops-02). Recorded as partially verified
HMS-NF-004 (3-year audit log retention)	Cannot be observed within one semester. Replaced by inspection of the retention setting and an archive/restore demonstration on seeded data (TC-Ops-01). Recorded as partially verified
Browser/OS compatibility beyond current Chrome, Firefox and Edge on Windows and Linux, plus Chrome on Android	Limited lab devices. Safari and iOS are not available to the team
Load beyond 200 concurrent users, or data volumes beyond 5,000 students and 2,000 rooms	Targets stated in HMS-NF-001 and HMS-NF-006 are the ceiling for this project
Penetration testing of any host not owned by the team	Only the team's local and staging deployments are tested. No scanning of university or gateway infrastructure

Excluded item	Reason for exclusion
Technician sub-role and the optional DRF REST API	Marked optional in the SRS. Tested only if implemented in Sprint 3; otherwise the related requirements stay at status N with this justification
Usability testing with external end users (wardens, students outside the team)	No customer access within the course timeline. Replaced by a heuristic and accessibility review by a team member who did not build the screen

5. Test Approach / Strategy

Development follows the Agile lifecycle set out in the Phase 2 Project Plan, so testing is continuous rather than a single phase at the end. Unit tests are written alongside the code in each sprint and run on every push through Jenkins; integration, system, security and performance testing are run against the sprint build on the shared staging environment.

5.1 Security Validation

Security validation addresses the three security objectives in SRS Section 5.1.1 - confidentiality and access control, data protection, and integrity/auditability - and the five security requirements in SRS Section 5.1.2. Testing is a mix of configuration review, code and schema review, negative functional testing and tool-assisted scanning. All scanning is limited to the team's own local and staging deployments.

5.1.1 Planned security validation activities

Test case	Objective / requirement	What is done	Tools
TC-Sec-01	HMS-NF-003 - data protection	Review schema and code for any field holding card number, CVV or expiry; inspect database dump and application logs for personal data written in the clear; confirm personal data is only exposed to the owning student or an Admin	psql, grep, manual code review, SonarQube
TC-Sec-02	HMS-SR-001 - transport security	Scan the staging endpoint for negotiated TLS versions and ciphers; confirm TLS 1.0/1.1 are refused; verify plain HTTP returns a 301 to HTTPS and that SECURE_SSL_REDIRECT, HSTS and secure cookie settings are enabled in Django settings	testssl.sh, browser dev tools, Django check --deploy
TC-Sec-03	HMS-SR-002 - password storage	Create a user, inspect the stored hash prefix (pbkdf2_sha256\$ or argon2), confirm salt differs for identical passwords, confirm no password appears in logs, fixtures or error pages	psql, pytest, grep over logs
TC-Sec-04	HMS-SR-003, HMS-F-005 - access control	Vertical escalation: a Student session requests every Admin/Warden URL and, if the API exists, every admin endpoint with a student token - expect 403. Horizontal escalation (IDOR): request another student's profile, fee statement, receipt and complaint by changing the ID - expect 403/404. Forced browsing of unlinked admin URLs. Verify each denial is written to the audit log	Postman/newman, OWASP ZAP authenticated scan, pytest
TC-Sec-05	HMS-SR-004 - auditability	Perform a sample set of admin actions (allocation change, fee-structure edit, complaint status change);	pytest, psql

Test case	Objective / requirement	What is done	Tools
		verify one audit entry each with actor, timestamp, action and target; confirm the application exposes no path to edit or delete audit rows, and that entries survive an application restart	
TC-Sec-06	HMS-SR-005 - payment scope	Confirm the payment flow hands off to the gateway-hosted page or gateway SDK and that HMS receives only a token/order reference; grep the schema and codebase for card, cvv and pan fields; confirm no card data in request logs	Manual review, Bandit, grep
TC-Sec-07	Injection defence (supports HMS-NF-003)	SQL injection and stored/reflected XSS payloads submitted through complaint description, profile fields, report date filters and search boxes; verify ORM parameterisation and template auto-escaping; confirm payloads are stored and rendered inert	OWASP ZAP active scan, manual payload list
TC-Sec-08	Session and CSRF handling (supports HMS-F-004)	POST without or with a stale CSRF token expects 403; session cookie carries HttpOnly, Secure and SameSite; session identifier is rotated on login (no fixation); idle timeout at 20 minutes and logout invalidates the session server-side	Browser dev tools, ZAP, pytest
TC-Sec-09	File upload safety (supports HMS-F-030)	Complaint attachment restricted by extension, MIME type and size; upload of an executable, a script-in-image and an oversized file is rejected; stored files are served without execution and cannot be reached by path traversal	Manual negative tests, curl
TC-Sec-10	Payment webhook integrity (supports HMS-F-021, HMS-F-024)	Webhook signature verification: an unsigned or wrongly signed callback is rejected; a captured valid callback replayed twice credits the student only once (idempotency); amount tampering in the callback is detected	Postman replay, pytest
TC-Sec-11	Dependency and static analysis	Dependency vulnerability scan of requirements.txt and static security analysis of the codebase, run as a Jenkins stage; Blocker and Critical findings block the merge	pip-audit, Bandit, SonarQube, Jenkins
TC-Sec-12	Brute force and OTP abuse (supports HMS-F-001, HMS-F-003)	Repeated failed logins and repeated OTP requests for one account are throttled and logged; OTP is single-use and expires at 10 minutes; a used OTP cannot be replayed	Locust, pytest, manual

5.1.2 Notes on security scope

- TC-Sec-01 to TC-Sec-06 verify the security requirements named in the SRS. TC-Sec-07 to TC-Sec-12 are additional validation drawn from the OWASP Top 10 that the SRS does not yet state as requirements; they are listed here as planned activities and are proposed as new security requirements in SRS v1.1 (see Section 13.3).
- Security testing is planned, not exploratory only: each case has a defined expected result, and any finding is logged as a defect with a severity band from Section 14.3.
- A Severity 1 or 2 security defect is a release blocker under the exit criteria in Section 5.5, regardless of functional test results.

- No test data contains real student personal information. All accounts, cards and contact details are synthetic.

5.2 Test Levels

Level	Scope and technique	Tools	Primary owner
Unit	Models, form and serializer validation, service functions, capacity and fee calculations, permission decorators. White-box; statement and branch coverage measured. Target: 70% branch coverage overall, 80% for AllocationModule and FeeModule	pytest, pytest-django, coverage.py	Each module owner writes unit tests for their own module
Integration	Interfaces between Django apps (allocation to inventory, fee to audit, complaint to notification) and between HMS and external services (gateway sandbox, mock SMTP). Run against a real PostgreSQL instance so constraints and transactions are exercised	pytest + Django test client, Postman/newman, MailHog, gateway sandbox	Module owner of the calling side, reviewed by the owner of the called side
System	End-to-end journeys per use case through the browser, covering both actors in SRS Section 7.1. Black-box. Includes negative paths and the error messages named in the acceptance criteria	Selenium WebDriver with pytest, manual execution for the course test-case template	Cross-assigned (see Section 9.2)
Acceptance (UAT)	Scripted walkthrough of the eight acceptance suites in SRS Section 6, executed against the demo build with a team member acting as Warden and another as Student	Manual checklist, screenshots	Test Lead, witnessed by the whole team
Regression	Full unit and integration suite on every pull request in Jenkins; system smoke suite (login, allocate, pay, raise complaint, report) before every demo and after every hotfix	Jenkins pipeline	Test Lead maintains the pipeline

5.3 Test Types

- Functional testing of all 20 functional requirements, positive and negative paths.
- Database and data-integrity testing: unique and foreign-key constraints, the capacity constraint behind HMS-F-015, and atomicity of allocation and payment updates under concurrent requests.
- Regression testing on every merge, as above.
- Performance and scale testing against HMS-NF-001 and HMS-NF-006.
- Security validation as detailed in Section 5.1.
- Usability and accessibility testing against HMS-NF-005, including a keyboard-only pass and contrast checks.
- Static analysis and code review: SonarQube quality gate, flake8, Bandit, and a mandatory peer review on every pull request as required by the course guidelines.

5.4 Test Design Techniques

Test cases are derived deliberately rather than ad hoc. The technique used for each area is recorded so that the 8-10 cases per use case in the Test Case Specification are justified rather than padded.

Technique	Where it is applied	Examples
Equivalence partitioning	Fee amounts, room types, complaint categories, report date ranges, roll-number formats	Valid institute roll number / malformed / already registered
Boundary value analysis	Room capacity, OTP expiry, session timeout, report ranges, upload size	Occupancy at N-1, N and N+1 for HMS-F-015; OTP validated at 9:59, 10:00 and 10:01 minutes; session at 19 and 21 minutes
Decision table	Allocation eligibility in HMS-F-012, combining room availability, gender policy match and stated preference	8 combinations of the three conditions, each with the expected allocate/reject outcome and message
State transition testing	Complaint lifecycle (Open, In Progress, Resolved) and room-change request (Pending, Approved, Rejected)	Valid transitions succeed and are timestamped; invalid transitions such as Resolved back to Open are refused
Use-case based testing	The two use-case diagrams in SRS Section 7.1 drive the system-level journeys	Student: register, view allocation, request change, view fees, pay, download receipt, raise complaint, update profile
Error guessing and negative testing	Concurrency and data-integrity edges	Two admins allocating the last bed in a room at the same time; duplicate roll-number registration; webhook replay; back-button resubmission of a payment

5.5 Entry and Exit Criteria

5.5.1 Entry criteria (testing may begin when all hold)

- SRS v1.0 is baselined and the RTM in Section 13 is populated for the features in the build.
- The feature branch is merged to develop and the Jenkins build, including unit tests and the SonarQube gate, is green.
- A smoke test - log in as Student and as Admin, and load both dashboards - passes on the staging build.
- Test data is seeded: hostel blocks, rooms, students, one term fee structure, and a set of open complaints.
- Gateway sandbox credentials and the mock SMTP catcher are reachable from staging.
- Test cases for the features in the build are written, reviewed and linked to requirement IDs.

5.5.2 Exit criteria (testing is complete when all hold)

- 100% of planned test cases executed, with at least 95% passing.
- No open Severity 1 or Severity 2 defects, including security defects.
- Every High-priority functional requirement and all five security requirements pass their mapped test cases.

- Branch coverage at or above 70% overall and 80% for AllocationModule and FeeModule, evidenced by the coverage report.
- SonarQube reports no new Blocker or Critical issues and no unreviewed security hotspots.
- HMS-NF-001 met at p95 ≤ 2 s for 200 concurrent users; HMS-NF-005 audit shows no Severity 1 or 2 accessibility failures.
- Every requirement in the RTM carries status A, or status N/P with a written justification from Section 4.
- UAT walkthrough completed and signed off in Section 15.

6. Test Environment

6.1 Environments

Environment	Purpose	Configuration
Local development (one per member)	Unit tests, test-case authoring, debugging	Docker Compose: Django dev server, PostgreSQL 15, MailHog. SQLite is not used, so that PostgreSQL-specific constraints are exercised
CI (Jenkins agent)	Unit and integration suites, coverage, static and dependency analysis on every push and pull request	Ephemeral containers built from the repository Dockerfile; PostgreSQL service container; no external network calls except the package index
Staging	Integration, system, security, performance and accessibility testing; UAT	Single shared Linux VM: Nginx with TLS, Gunicorn, PostgreSQL, gateway sandbox keys, seeded data. Mirrors the intended production topology from SRS Section 2.4
Demo	Final demonstration build	Snapshot of the last passing staging build with presentation-friendly data

6.2 Software and Tools

Purpose	Tool
Application stack under test	Python 3.11, Django 5.x, Django REST Framework (optional), PostgreSQL 15, Gunicorn, Nginx, Bootstrap 5
Unit and integration testing	pytest, pytest-django, factory_boy for test data, freezegun for time-dependent cases (OTP and session expiry)
Coverage	coverage.py, reported as statement and branch coverage per Django app
UI and system testing	Selenium WebDriver with ChromeDriver and GeckoDriver, pytest-selenium
API testing	Postman collections, executed in CI through newman
Performance and scale	Locust for concurrent-user load, Django Debug Toolbar and django-silk for query profiling, pgbench-style seeded data for scale runs
Security	OWASP ZAP (authenticated scan), testssl.sh, Bandit, pip-audit, SonarQube security hotspots, Django check --deploy
Accessibility	axe DevTools browser extension, Lighthouse, manual keyboard-only navigation, browser responsive-design mode at 360, 768 and 1366 px

Purpose	Tool
CI/CD and containers	Jenkins pipeline, Docker and Docker Compose
Static analysis and code quality	SonarQube, flake8, black (formatting check)
Defect and work tracking	GitHub Issues with severity and module labels, linked to the GitHub Project board used for the sprint backlog
Email and OTP capture	MailHog as a local SMTP catcher
Test evidence	Screenshots, Selenium HTML reports, coverage HTML report, ZAP report, Locust report, all committed under /docs/test-evidence/

6.3 Test Data

- Seeded through a Django management command (seed_testdata) so that any member can rebuild an identical dataset, and so the same data is used in CI and on staging.
- Baseline functional dataset: 4 hostel blocks, 120 rooms of mixed single and shared type, 300 students, one term fee structure, 40 complaints across all statuses, 20 fee defaulters.
- Scale dataset for HMS-NF-006 and TC-Scale-01: 5,000 students and 2,000 rooms.
- Negative and boundary data: malformed roll numbers, full rooms, expired OTPs, zero and negative fee amounts, oversized and wrong-type complaint attachments, tampered webhook payloads.
- Payment data: only the test card numbers published in the gateway sandbox documentation. No real card data is used or stored at any point.
- Personal data: entirely synthetic names, emails and phone numbers. No real student records are loaded into any environment.

7. Test Schedule

Dates below align to the sprint boundaries in the Phase 2 Project Plan and are to be reconciled with the final course submission dates.

Milestone	Start	End	Owner
Test plan baselined and reviewed	16-09-2026	22-09-2026	Test Lead
Test case specification: 8-10 cases per implemented use case	23-09-2026	03-10-2026	All members, own modules
Test case peer review and linking to requirement IDs	04-10-2026	06-10-2026	Cross-assigned pairs
Staging environment and seeded test data ready	01-10-2026	05-10-2026	Arjun (env), Arnav (seed data)
Unit testing and coverage (continuous, per sprint)	28-09-2026	31-10-2026	All members
Integration testing	12-10-2026	24-10-2026	Module owners in pairs
System testing on the Sprint 3 build	26-10-2026	07-11-2026	Cross-assigned pairs
Security validation (Section 5.1)	02-11-2026	08-11-2026	Arnav, with full-team review

Milestone	Start	End	Owner
Performance, scale and accessibility testing	09-11-2026	12-11-2026	Arjun (perf), Arjav (a11y)
Defect fixing and regression cycles	13-11-2026	16-11-2026	All members
UAT walkthrough and sign-off	17-11-2026	19-11-2026	Whole team
Test summary report and final RTM update	20-11-2026	21-11-2026	Test Lead

8. Test Deliverables

Deliverable	Description	Owner
Software Test Plan	This document, version controlled in the repository	Test Lead
Test Case Specification	8-10 unit, integration and system test cases per implemented use case, in the course test-case template (Test Case ID, Module, Description, Pre-conditions, Steps, Test Data, Expected Result, Actual Result, Test Result)	All members
Automated test suites	pytest unit and integration suites, Selenium system suite, Postman collection, Locust scenario file	All members
Test data scripts	seed_testdata management command and fixture files	Arnav
Execution logs and evidence	Jenkins run logs, Selenium HTML reports, screenshots for manual cases	All members
Coverage report	coverage.py HTML and summary showing statement and branch coverage per module	Test Lead
Security validation report	Results of TC-Sec-01 to TC-Sec-12, ZAP and testssl output, Bandit and pip-audit findings with dispositions	Arnav
Performance report	Locust results against HMS-NF-001 and scale results against HMS-NF-006	Arjun
Defect reports	GitHub Issues with severity, module, steps to reproduce and resolution	All members
Requirements Traceability Matrix	Section 13, updated to final status at the end of execution	Test Lead
Test Summary Report	Metrics from Section 14, exit-criteria assessment, residual risk and known limitations	Test Lead
Maintenance plan input	Known defects, quarantined tests and deferred items handed to the Phase 4 maintenance plan	Test Lead

9. Roles and Responsibilities

9.1 Assignments

Role	Name	Responsibility
Test Lead (this phase)	Arnav Sharath Hanchanur	Owns this plan, the RTM, metrics and the test summary report; maintains the Jenkins test stages; leads security validation

Role	Name	Responsibility
Module owner and tester - AuthModule	Arjav Chandranath	Unit and integration tests for HMS-F-001 to HMS-F-005; TC-Sec-03; accessibility audit (TC-UX-01)
Module owner and tester - AllocationModule	Arjun Mohan Gowda	Unit and integration tests for HMS-F-010 to HMS-F-015, including the capacity decision table and concurrency cases; staging environment; performance and scale runs
Module owner and tester - FeeModule	Armaan Ali	Unit and integration tests for HMS-F-020 to HMS-F-024; gateway sandbox integration; webhook integrity (TC-Sec-10)
Module owner and tester - Complaint and Reporting modules	Arnav Sharath Hanchanur	Unit and integration tests for HMS-F-030, HMS-F-031, HMS-F-040, HMS-F-041; file-upload safety (TC-Sec-09); seed data
Defect triage	Whole team, weekly	Assign severity and owner, confirm fixes, decide deferrals; disagreements escalated to the Course Coordinator
Reviewer / approver	Course Coordinator	Reviews this plan and the test summary report; approves readiness for demonstration

9.2 Cross-Testing Rule

- No member executes the system-level or acceptance tests for a module they implemented. System testing is swapped: Arjav tests Allocation, Arjun tests Auth, Armaan tests Complaints and Reporting, Arnav tests Fees.
- Every test case is reviewed by one member other than its author before execution, in the same pull request as the code it covers.
- Defects are logged by the tester, not the implementer, and closed only after the tester re-runs the case.

10. Risks and Mitigation

Risk	Likelihood / Impact	Mitigation
Payment gateway sandbox credentials delayed, throttled or changed mid-project	Medium / High	Put the gateway behind a thin adapter interface with a mock implementation; record real sandbox responses as fixtures so integration tests can run offline
Allocation concurrency defects (two students placed in the last bed) surface only at system test	Medium / High	Write the database capacity constraint and a concurrent-allocation test in Sprint 2, before the UI is built; use <code>select_for_update</code> and test it explicitly
Coverage and SonarQube gates start failing close to the deadline	High / Medium	Enforce both gates from Sprint 1 so coverage debt never accumulates; block merges rather than allowing exceptions
Contention for the single shared staging VM during performance runs	Medium / Medium	Book staging slots in the sprint plan; fall back to a Dockerised local instance for load runs, recording that results are indicative
Uneven contribution across the team - commits and individual weekly evaluation are checked	Medium / High	Module ownership table in Section 9, cross-testing rule in 9.2, and a weekly RTM and commit review at the sprint meeting

Risk	Likelihood / Impact	Mitigation
Selenium test flakiness inflating the failure count and hiding real defects	High / Low	Explicit waits instead of sleeps, one automatic retry, and a quarantine list reported separately in the metrics rather than mixed into pass rate
Scope creep from optional items (REST API, technician sub-role) diluting test effort	Medium / Medium	Test optional items only if they ship; otherwise their requirements stay at status N with the justification from Section 4
Late SRS changes invalidating written test cases	Medium / Medium	Test cases reference requirement IDs, not prose; any SRS change is recorded in the SRS revision history and the affected rows of the RTM are re-reviewed
Real student data used in test environments	Low / High	Synthetic seed data only, generated by script; no production or university dataset is imported at any stage
Team unfamiliar with a tool (ZAP, Locust, Jenkins) losing time during the test window	Medium / Medium	One spike per tool in Sprint 1 or 2, with the owner writing a short runbook in the repository

11. Assumptions and Dependencies

11.1 Assumptions

- SRS v1.0 is the agreed baseline; changes arrive through its revision history and not verbally.
- The sandbox payment gateway behaves like production for order creation, redirect and webhook delivery, and its test cards remain valid for the project window.
- The staging VM stays available to the team for the full execution window.
- A team member standing in as Warden is an acceptable substitute for a real end user during UAT, given that no external customer is available.
- Requirements marked optional in the SRS are out of scope unless implemented and declared before system testing begins.

11.2 Dependencies

- Gateway sandbox account and API keys, needed before integration testing of HMS-F-021 can start.
- TLS certificate (self-signed is acceptable for staging) required before TC-Sec-02 can be executed.
- Jenkins access and a configured agent, required for the coverage and static-analysis gates.
- Architecture and Design Specification DS-* identifiers, required to complete the design reference column of the RTM.
- Course Coordinator review of this plan, required before execution formally starts.

12. Suspension and Resumption Criteria

12.1 Suspend testing condition

- The staging environment or the database is unavailable for more than 4 hours.

- The build blocks more than 30% of the planned test cases - for example login is broken, so no authenticated case can run.
- A Severity 1 defect is open in the module under test, making further results meaningless.
- A Severity 1 security defect is found (for example RBAC is not enforced at the API layer, or card data is being stored); in that case testing of the affected module stops and the defect is escalated immediately.
- The gateway sandbox is unreachable for more than one working day and the mock adapter cannot substitute for the case in hand.
- Test data is corrupted or found to contain real personal data.

12.2 Resume testing when

- The blocking defect is fixed, merged and verified by the tester who raised it.
- The environment is restored and the smoke suite (login, allocate, pay, raise complaint, report) passes.
- Test data has been rebuilt from the seed script and verified.
- Any test cases invalidated by the fix have been re-reviewed and re-executed, not only the failing case.

13. Test Case Management and Traceability

13.1 Test Case Identification Convention

The SRS names one anchor test case per requirement (for example TC-Auth-01). Each anchor expands into 8-10 detailed cases in the Test Case Specification, using the course convention of a level prefix and a sequence number, and every detailed case carries its anchor and requirement ID in its own row:

Prefix	Meaning	Example
UT_nn	Unit test case	UT_07 - allocate_room() rejects a student whose gender does not match the block policy (HMS-F-012, anchor TC-Alloc-03)
IT_nn	Integration test case	IT_04 - successful gateway webhook reduces dues and writes an audit entry (HMS-F-021, anchor TC-Fee-02)
ST_nn	System test case	ST_11 - student pays fees end to end and downloads the PDF receipt (HMS-F-021 and HMS-F-023)
SEC_nn	Security test case	SEC_03 - student token calling an admin endpoint returns 403 and is logged (HMS-SR-003, anchor TC-Sec-04)
PERF_nn	Performance test case	PERF_01 - 200 concurrent users on the dashboard, p95 under 2 s (HMS-NF-001, anchor TC-Perf-01)

Test cases live in the repository under /tests and /docs/test-cases so that they are version controlled alongside the code, and defects are raised as GitHub Issues that reference the failing test case ID and requirement ID.

13.2 Requirements Traceability Matrix

Status legend: N - not tested, P - partially tested, A - all mapped test cases passed. Status is updated at the end of each execution cycle; the values below are the position at the time of planning.

Req ID	Requirement (short)	Design ref	Module	Test case(s)	St.
HMS-F-001	Register / login with role assignment	DS-Auth-01	AuthModule	TC-Auth-01 (UT_01-03, IT_01, ST_01-02)	N
HMS-F-002	Student profile update	DS-Auth-03	AuthModule	TC-Auth-02 (UT_04, ST_03)	N
HMS-F-003	OTP password reset, 10-min expiry	DS-Auth-04	AuthModule	TC-Auth-03 (IT_02, ST_04), TC-Sec-12	N
HMS-F-004	Session timeout after 20 min	DS-Auth-05	AuthModule	TC-Auth-04 (ST_05), TC-Sec-08	N
HMS-F-005	Enforce RBAC	DS-Auth-02	AuthModule	TC-Auth-05, TC-Sec-04 (SEC_01-04)	N
HMS-F-010	View own room and roommates	DS-Alloc-03	AllocationModule	TC-Alloc-01 (UT_05, ST_06)	N
HMS-F-011	Admin CRUD on student records	DS-Alloc-04	AllocationModule	TC-Alloc-02 (UT_06, IT_03)	N
HMS-F-012	Allocate / reallocate room	DS-Alloc-01	AllocationModule	TC-Alloc-03 (UT_07-10, IT_04, ST_07)	N
HMS-F-013	Room change request and approval	DS-Alloc-05	AllocationModule	TC-Alloc-04 (IT_05, ST_08)	N
HMS-F-014	Manage blocks, rooms and capacity	DS-Alloc-06	AllocationModule	TC-Alloc-05 (UT_11, IT_06)	N
HMS-F-015	Prevent over-allocation	DS-Alloc-02	AllocationModule	TC-Alloc-06 (UT_12-14 BVA, IT_07 concurrency)	N
HMS-F-020	Itemised fee statement	DS-Fee-03	FeeModule	TC-Fee-01 (UT_15, ST_09)	N
HMS-F-021	Online fee payment (sandbox)	DS-Fee-01	FeeModule	TC-Fee-02 (IT_08-09, ST_10), TC-Sec-10	N
HMS-F-022	Configure fee structure per term	DS-Fee-04	FeeModule	TC-Fee-03 (UT_16, IT_10)	N
HMS-F-023	PDF payment receipt	DS-Fee-05	FeeModule	TC-Fee-04 (IT_11, ST_11)	N
HMS-F-024	Reconcile payments / flag defaulters	DS-Fee-02	FeeModule	TC-Fee-05 (IT_12, ST_12)	N
HMS-F-030	Raise maintenance complaint	DS-Maint-01	ComplaintModule	TC-Maint-01 (UT_17, ST_13), TC-Sec-09	N
HMS-F-031	Complaint status workflow	DS-Maint-02	ComplaintModule	TC-Maint-02 (IT_13, ST_14 state transitions)	N
HMS-F-040	Generate occupancy / fee reports	DS-Rep-01	ReportingModule	TC-Rep-01 (IT_14, ST_15)	N
HMS-F-041	Export reports as CSV and PDF	DS-Rep-02	ReportingModule	TC-Rep-02 (ST_16)	N
HMS-NF-001	p95 response time $\leq$ 2 s at 200 users	DS-Perf-01	WebUI / all modules	TC-Perf-01 (PERF_01-02)	N
HMS-NF-002	99.5% availability per semester	DS-Ops-01	Deployment config	TC-Ops-02 (soak test + config review) - partial, see Section 4	N

Req ID	Requirement (short)	Design ref	Module	Test case(s)	St.
HMS-NF-003	Data protection, no card data stored	DS-Sec-02	All modules	TC-Sec-01, TC-Sec-06, TC-Sec-07	N
HMS-NF-004	3-year audit log retention	DS-Ops-02	Audit component	TC-Ops-01 (config inspection + archive/restore) - partial, see Section 4	N
HMS-NF-005	Responsive UI and WCAG 2.1 AA	DS-UX-01	WebUI	TC-UX-01 (manual audit at 3 viewports, keyboard pass)	N
HMS-NF-006	Scale to 5,000 students / 2,000 rooms	DS-Scale-01	All modules	TC-Scale-01 (PERF_03)	N
HMS-SR-001	TLS 1.2+ mandatory, HTTP redirected	DS-Sec-03	Deployment config	TC-Sec-02	N
HMS-SR-002	Salted password hashing	DS-Sec-01	AuthModule	TC-Sec-03 (UT_18)	N
HMS-SR-003	RBAC at view and API layer	DS-Sec-04	AuthModule / API layer	TC-Sec-04 (SEC_01-04)	N
HMS-SR-004	Immutable audit log of admin actions	DS-Sec-05	Audit component	TC-Sec-05 (SEC_05-06)	N
HMS-SR-005	PCI-DSS gateway only, no card data	DS-Sec-06	FeeModule	TC-Sec-06	N

13.3 Traceability Gaps and Actions

Reviewing the SRS against this plan surfaced the following items. They are recorded here so that the next SRS revision closes them rather than leaving the traceability one-directional.

Observation	Action
The SRS RTM (Section 8) lists only 10 of the 31 requirements, so 21 requirements had no test mapping.	Section 13.2 of this plan extends the RTM to all 31 requirements. Replace the SRS RTM with this version in SRS v1.1
The SRS synopsis in Section 4 states 20 functional requirements in one sentence and 18 in the next, and Section 7.1 repeats 18. The actual count across Sections 4.1 to 4.5 is 20 (5 + 6 + 5 + 2 + 2).	Correct the counts to 20 in SRS v1.1. This plan uses 20 throughout
The SRS Section 4 synopsis refers to six feature groups, but only five are specified (4.1 to 4.5).	Correct to five feature groups, or add the missing group, in SRS v1.1
The two use-case diagrams in SRS Section 7.1 are said to cover all functional requirements, but HMS-F-003, HMS-F-004, HMS-F-005, HMS-F-015 and HMS-F-041 do not appear as use cases.	Either add them to the diagrams or state that they are non-actor-initiated requirements verified by test rather than by use case. This plan tests all of them regardless
TC-Sec-07 to TC-Sec-12 in Section 5.1 validate behaviours (injection defence, CSRF and session handling, upload safety, webhook integrity, dependency scanning, brute-force throttling) that no SRS requirement currently states.	Add matching security requirements HMS-SR-006 onwards in SRS v1.1 so the security testing is traceable in both directions

Observation	Action
HMS-NF-002 and HMS-NF-004 cannot be fully verified within the project timeline.	Keep the requirements as written, but record them as partially verified with the substitute checks in Section 4 and the RTM

14. Test Metrics and Reporting

14.1 Metrics Collected

Metric	Definition	Target
Test case execution rate	Cases executed / cases planned	100% by the end of the execution window
Pass rate	Cases passed / cases executed	>= 95%
Requirement coverage	Requirements with at least one executed test case / total requirements	100% of the 31 requirements
Branch coverage	Branches exercised / total branches, from coverage.py	>= 70% overall; >= 80% AllocationModule and FeeModule
Defect density	Defects found / KLOC, per module	Tracked for comparison between modules, no fixed target
Defect severity distribution	Count of open and closed defects by severity band	0 open Severity 1 or 2 at exit
Defect aging	Mean and maximum days a defect stays open, by severity	Severity 1 within 1 day, Severity 2 within 3 days
Defect removal efficiency	Defects found before UAT / total defects found	>= 90%
Flaky test count	Cases that passed on retry without a code change	Reported separately, kept below 5% of the automated suite
Security findings	Open findings from ZAP, Bandit, pip-audit and SonarQube hotspots, by severity	0 High or Critical open at exit

14.2 Reports

- Automated per-commit report: Jenkins build status with unit and integration results, coverage delta and SonarQube gate outcome, visible on every pull request.
- Weekly status, at the sprint meeting: execution and pass rate, new and closed defects by severity, RTM status changes, and blockers - one page.
- Security validation report at the end of the security window: results of TC-Sec-01 to TC-Sec-12 with tool output and the disposition of each finding.
- Test Summary Report at the end of execution: final metrics, exit-criteria assessment, list of deferred defects and known limitations, and the residual risk statement handed to the maintenance plan.

14.3 Defect Severity Bands

Severity	Definition	Examples	Response
1 - Critical	Core functionality unusable, data loss or corruption, or a security control absent	Two students allocated to one bed; card data found in the database; RBAC not enforced at the API layer	Testing of that module suspended; fix within 1 working day
2 - High	A High-priority requirement fails with no workaround	Payment succeeds at the gateway but dues are not updated; report totals do not match records	Fix within 3 working days; release blocker
3 - Medium	Requirement fails but a workaround exists, or a Medium-priority requirement fails	Receipt PDF missing the transaction ID; complaint notification not sent though status updates correctly	Fix within the sprint
4 - Low	Cosmetic, wording, or minor usability issue	Misaligned table on a 360 px viewport; unclear validation message	Fix if time permits; otherwise deferred and listed in the summary report

15. Approvals

This plan is in effect once reviewed by the team and approved by the Course Coordinator. The same table is signed again against the Test Summary Report at the end of execution.

Role	Name	Signature	Date
Team member	Arnav Sharath Hanchanur		
Team member	Arjav Chandranath		
Team member	Arjun Mohan Gowda		
Team member	Armaan Ali		
Course Coordinator			